MENLO SCHOOL WHITAKER LAB

December 12, 2025

Applied Science Research Great Wall Paper (Modified)

Ethan Shapero

Abstract

A greenhouse automation system was designed to control all systems in Menlo's new greenhouse, ensuring the climate inside matches the parameters set by teachers or students. The system senses soil moisture, temperature, humidity, and CO2, and then activates solenoids, valves, exhaust fans, and heaters depending on the readings. To date, the main control box has been built and tested; however, eventually it will be installed in the greenhouse, where it will be powered by two large 400W solar panels and run indefinitely.

(To be submitted to Dr. Dann, Ph.D.)

1 History

Greenhouses have been around since 30 CE, when the Romans built a system that allowed the Roman emperor to eat his daily cucumber; the emperor really liked cucumbers and could not go without them for more than a day. The Romans planted a cucumber in a large wheeled box, and then every day they would wheel it into the sun, and every night they would wheel it back inside where it was warm. While this original setup is quite different from what people consider greenhouses today, the idea of modifying and idealizing a plant's conditions even when it is cold or dry stays the same. Greenhouses, as people know them today, did not exist until 1500, when Korea started using them to grow certain medicinal plants year-round.

2 Design

The first aspect of the design is the external box. This component was designed to fit two constraints: the 3D printer that would be used, and it was unknown exactly which components would need to exit the box. It is for this reason that the only properly sized holes on the box are on the front panel for the display and control switches. The sides of the box have large cutout pieces removed from them, and this allows the designer to create swappable panels based on the sensors and outputs required. The first of these panels is the soil moisture sensor input port array. This, like all the other components, is 3D printed and lines up 10 soil moisture sensors in a 5x2 array for easy access. The sensors are labeled based on the “zone” they correspond to, and the sensors can be easily removed for transport. Another panel holds the output plugs, which are XT30 connectors that connect directly to the relay board. The connectors are fed from one large XT90 connector on another panel, which will connect to the solar batteries. The final panel only holds a USB-C port for programming and a slot for sensor wires to come out. Everything is printed from ABS because of its durability and high temperature resistance, with the goal of extending the box’s lifespan.

The next part of the design is the wiring. The Arduino Giga R1 was used because of its immense number of analog read ports and its compatibility with the Arduino display shield. Furthermore, the Giga R1 has wifi capabilities and a dual chip that can both run a website and collect data at the same time. All of the sensors are wired into a protoboard on top of the Arduino, which distributes power to the vast majority of components. Based on the inputs given in the UI, the Arduino will trigger relays on a standard relay board that will activate various components like solenoid irrigation valves.

The final part of the design is the UI and code. There are 4 main display pages or views that the user can access at any given time, and they are cycled between via the switches on the box face. The first page is a home page, which displays the status of each zone and the most relevant sensors' data averages. This page does not have extensive text, making it power-efficient. The second page is used to set parameters; here, users can use the built-in touchscreen to raise and lower the soil moisture content of each zone and set the overall

humidity level. The third page is a debug page, which displays every data value and analog sensor reading, allowing developers to fine-tune sensors and check for bugs. The final, and most important, page is the graphing logger; this page displays a graph of all data that has been collected by the greenhouse sensors. It automatically scales to the time frame and variance, while simultaneously increasing the interval at which it takes data points to preserve storage availability. The graph tracks all sensors at once, is color-coded, and has been tested to work up to 28 hours in a row.

Figure 1a: CAD of greenhouse control box case.

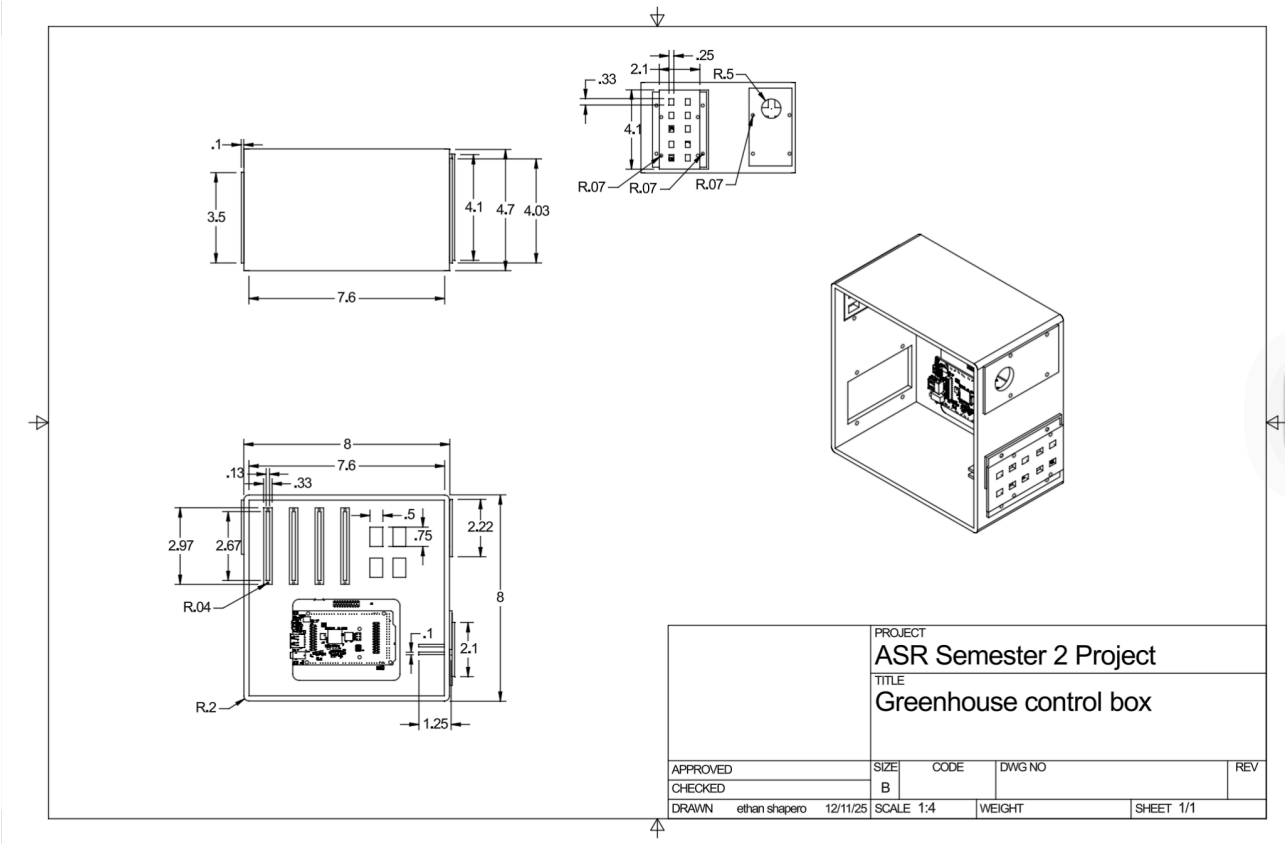


Figure 1b: Picture of greenhouse control box recording data at the end of the 28-hour stress test



3 How It Works

The heart of the automation system is the Arduino. As stated previously, the Arduino is hooked up to all of the different sensors and is the hub for all incoming and outgoing signals. The sensors all work differently, however, and so the Arduino has to interpret different inputs. The soil moisture sensors measure the resistance of an electrical signal through soil; The wetter the soil, the less resistance. This signal is then turned into an analog signal, which the Arduino picks up via its analog input pins. On the other hand, the temp/humidity sensor works via the I2C pin, which allows for communication between a wide variety of devices, and gives the output in actual degrees reading. The code accounts for these differences, running algorithms that convert the raw data into readable values. The code also runs a bias analysis software that detects and negates the values of sensors not in operation, or weighs them less depending on their inconsistency level. The Arduino then compares the estimated “real” value to the values set by the user, and if that value is higher/lower than the reading (by more than 5%), it will activate a mechanism to change that. For example, if the set soil moisture for zone 2 is 60%, but the soil moisture reading is only 45%, then the Arduino will activate a solenoid valve via the relay board to spray water for 10 seconds. Then, after a minute, if the reading is still below, the sprayer will activate again in a cycle that repeats until the soil moisture is at 60%. In this way, the greenhouse only uses the amount of water that is necessary, thus reducing water waste.

While the solar system is not installed yet, it is estimated that the system will draw a maximum of 20 amps at 24 volts, and a constant 0.5-1 amps. It is necessary to have ample energy storage, which will be a series of four 12V car batteries hooked up to be a 48V power bank (that is what the inverter requires). In theory, it should be able to run nonstop for years, without human intervention, and the greenhouse will ensure that the control box stays protected from the elements.

Figure 2a: Component diagram of greenhouse automation system

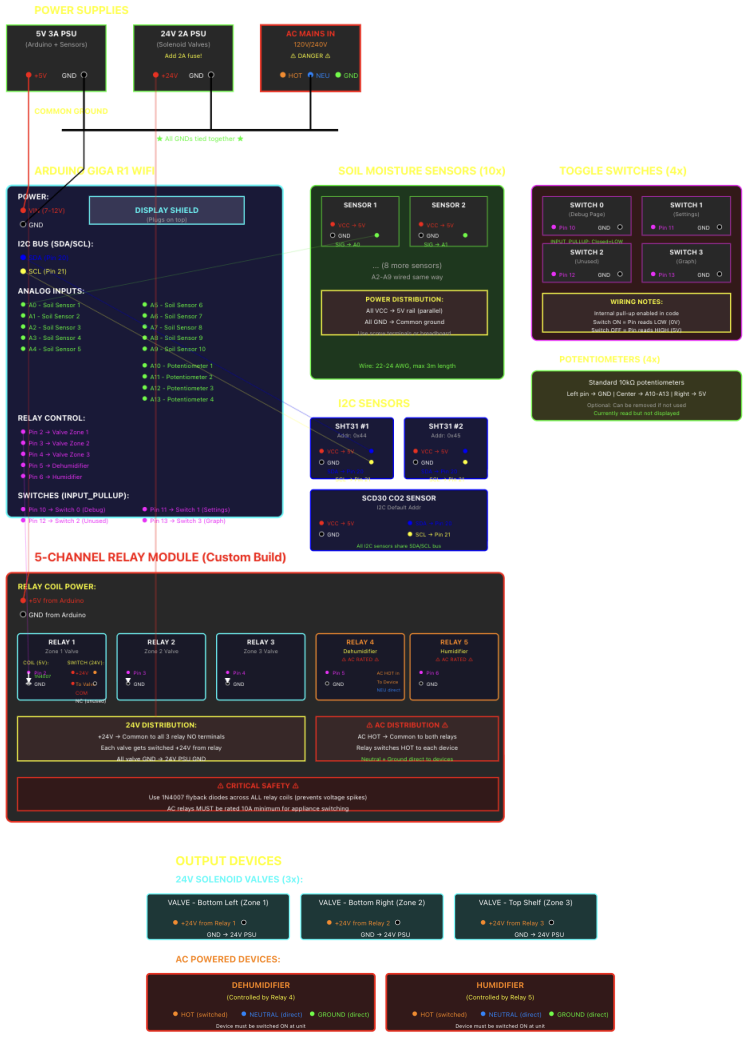
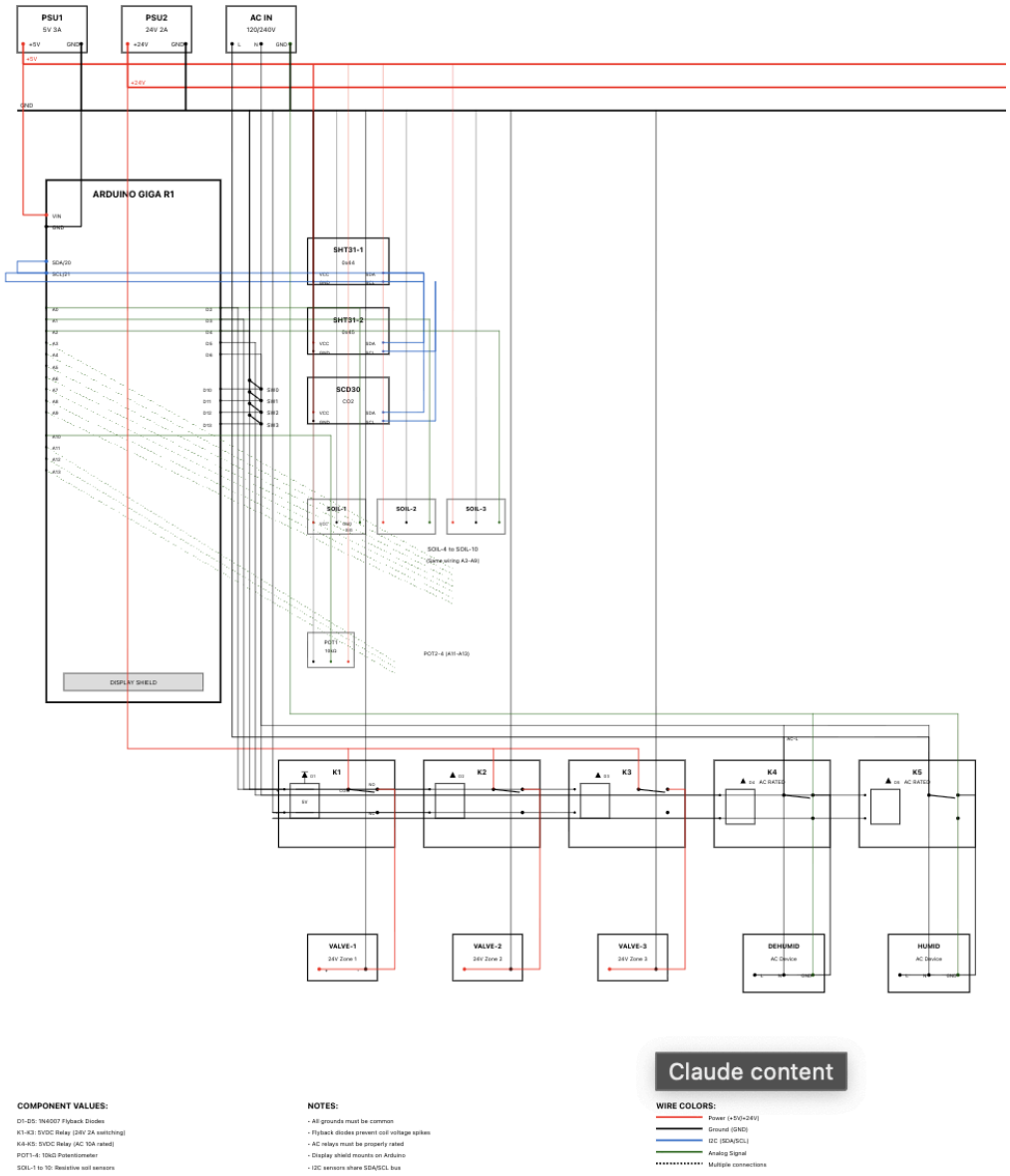


Figure 2b: Circuit Diagram of Greenhouse Control System



*Both Diagrams are made using AI but verified by the author due to complexity

4 Conclusion/Future Challenges

Overall, the greenhouse automation system works incredibly well. During its initial durability test, it operated 28 hours straight, collecting data on a patch of soil even when the sprinklers activated, spraying the box. It reliably tracks data and displays it in an intuitive and easy-to-operate manner. The separate external panels make upgrades easy, and the Arduino itself allows for the implementation of future features like online monitoring. While the system

may not technically meet code yet, it works reliably every time. The box will revolutionize environmental science at Menlo, allowing for research to be done in incredibly specific environments. The system will provide food and learning in a sustainable way that uses as few resources (like water and electricity) as necessary. Despite many challenges, such as broken sensors and overall complexity, the final product will be able to help Menlo for years to come.

In the final quarter, the box will need to be installed in the greenhouse itself. The greenhouse is still in development, and therefore, there is still more work to be done before the system can work at its best. Other challenges ahead include implementing the solar power system, testing for months rather than days, and integrating remote monitoring systems. There are not many more significant improvements beyond small refinements that could enhance the box itself; however, there is still a substantial amount of work to be done.

5 Bibliography

Claude AI, “Arduino Giga Soil Display Thread/Greenhouse Automation Project”, Private technical conversation, Oct.–Dec. 2025. [Online]. Available: https://claude.ai/share/d290918c-2ab4-480b-82f5-4b731b3abfd9 (accessed Dec. 12, 2025).

.

3B Sample Calculations

Power Draw

Measured wattage = 1.55w

Input voltage = 12v

P = IV

1.55w = I * 12v

I = .13 Amps

Solar Power

Rated Wattage = 400w

Rated Current = 10a

Rated Voltage = 40v

Estimated efficiency (sunny) = 95%

Estimated efficiency (partly cloudy) = 60%

Estimated efficiency (cloudy) = 25%

Actual Wattage for Both = Rated Wattage * Estimated Efficiacey * 2

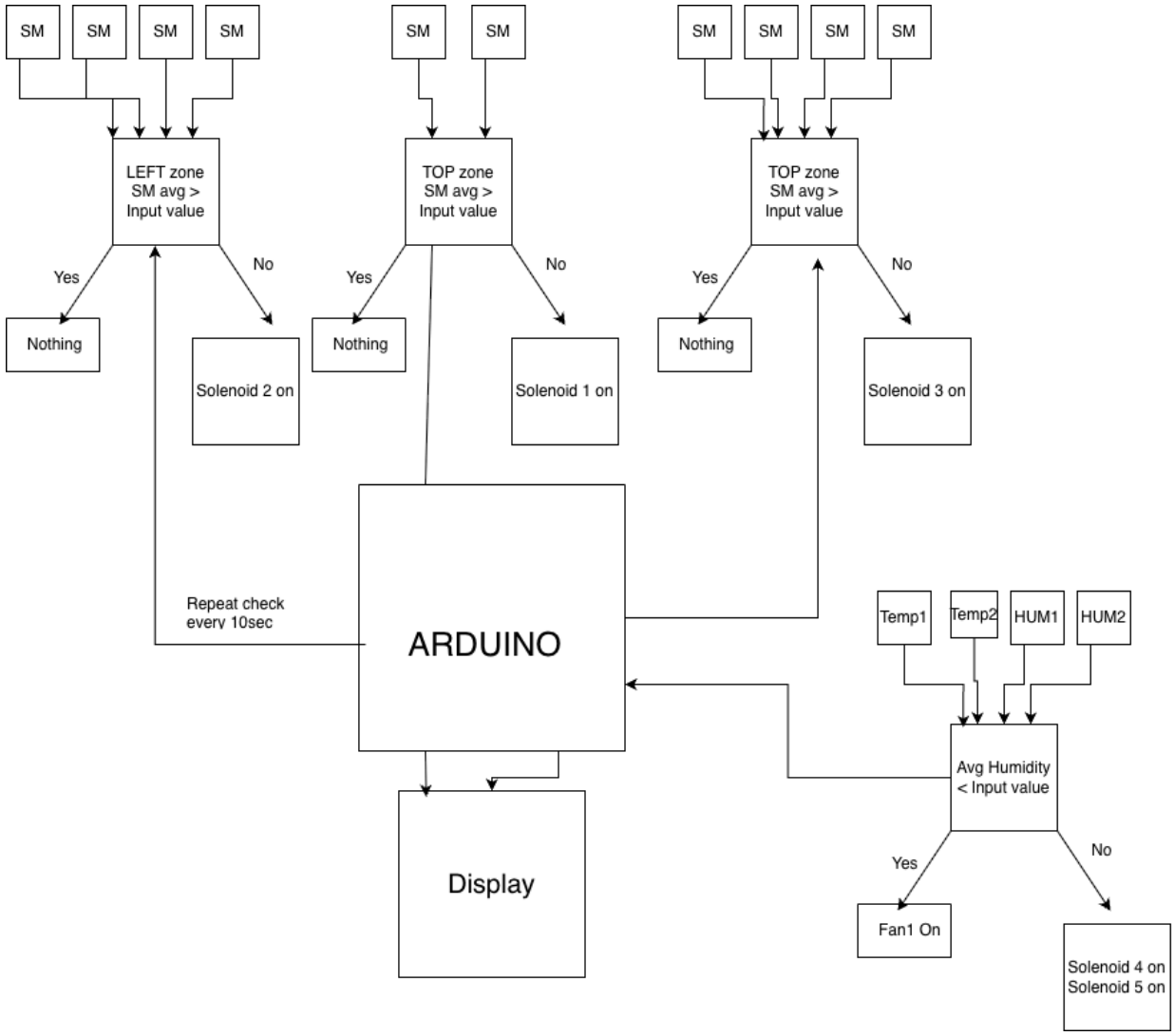
Actual Wattage (sunny) = 400w * 95% * 2 = 760w

Actual Wattage (partly cloudy) = 400w * 60% * 2 = 480w

Actual Wattage (cloudy) = 400w * 25% * 2 = 200w

3C Code

Figure 3A: Flow Chart of what code does



Code for Arduino Giga (Only part of it, total is 1500lines)

```
#include <Arduino_GigaDisplay.h>
#include <Arduino_GigaDisplay_GFX.h>
#include <Arduino_GigaDisplayTouch.h>
#include <Wire.h>
#include <Adafruit_SHT31.h>
#include <Adafruit_SCD30.h>

GigaDisplay_GFX tft;
Arduino_GigaDisplayTouch touchDetector;
Adafruit_SHT31 sht31_1 = Adafruit_SHT31();
```

```
Adafruit_SHT31 sht31_2 = Adafruit_SHT31();
Adafruit_SCD30 scd30;

// Colors
#define BG_COLOR        0x0000
#define TEXT_COLOR      0xFFFF
#define TITLE_COLOR     0x07FF
#define GOOD_COLOR      0x07E0  // Green
#define WARNING_COLOR   0xFFE0  // Yellow
#define BAD_COLOR       0xF800  // Red
#define VALVE_ON_COLOR  0x001F  // Blue
#define SET_COLOR       0xF81F  // Magenta
#define BUTTON_COLOR    0x4208  // Dark gray
#define BUTTON_PRESS    0x2104  // Darker gray

// Pins
#define VALVE_ZONE1     2
#define VALVE_ZONE2     3
#define VALVE_ZONE3     4
#define HUMID_HIGH_RELAY 5
#define HUMID_LOW_RELAY  6

// Timing constants
#define SPRAY_DURATION  10000  // 10 seconds
#define WAIT_DURATION   60000  // 60 seconds

// Pages
enum Page { STATUS_PAGE, SETTINGS_PAGE, DEBUG_PAGE, DIAGRAM_PAGE };
Page currentPage = STATUS_PAGE;
bool needsFullRedraw = true;

// Zone states
enum IrrigationState { IDLE, SPRAYING, WAITING };

struct Zone {
  float avgMoisture;
  float setMoisture;
  IrrigationState state;
  unsigned long stateStartTime;
  int valvePin;
  const char* name;
};

Zone zones[3] = {
  {0, 50, IDLE, 0, VALVE_ZONE1, "Bottom Left"},
  {0, 50, IDLE, 0, VALVE_ZONE2, "Bottom Right"},
  {0, 50, IDLE, 0, VALVE_ZONE3, "Top"}
};

float temp1, temp2, humidity1, humidity2, co2;
float setHumidity = 50;  // Default 50%
float potValues[4] = {0, 0, 0, 0};  // For debugging
int soilRawValues[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};  // Raw soil sensor values

// Button structure
```

```
struct Button {
  int x, y, w, h;
  const char* label;
  uint16_t color;
};

// Settings page buttons
Button zone1MinusBtn = {50, 110, 100, 70, "-", BUTTON_COLOR};
Button zone1PlusBtn = {650, 110, 100, 70, "+", BUTTON_COLOR};
Button zone2MinusBtn = {50, 210, 100, 70, "-", BUTTON_COLOR};
Button zone2PlusBtn = {650, 210, 100, 70, "+", BUTTON_COLOR};
Button zone3MinusBtn = {50, 310, 100, 70, "-", BUTTON_COLOR};
Button zone3PlusBtn = {650, 310, 100, 70, "+", BUTTON_COLOR};
Button humidMinusBtn = {50, 410, 100, 70, "-", BUTTON_COLOR};
Button humidPlusBtn = {650, 410, 100, 70, "+", BUTTON_COLOR};

void setup() {
  Serial.begin(115200);

  analogReadResolution(12);

  // Setup valve pins
  pinMode(VALVE_ZONE1, OUTPUT);
  pinMode(VALVE_ZONE2, OUTPUT);
  pinMode(VALVE_ZONE3, OUTPUT);
  pinMode(HUMID_HIGH_RELAY, OUTPUT);
  pinMode(HUMID_LOW_RELAY, OUTPUT);

  // Setup switch pins with INPUT_PULLUP
  pinMode(10, INPUT_PULLUP);
  pinMode(11, INPUT_PULLUP);
  pinMode(12, INPUT_PULLUP);
  pinMode(13, INPUT_PULLUP);

  digitalWrite(VALVE_ZONE1, LOW);
  digitalWrite(VALVE_ZONE2, LOW);
  digitalWrite(VALVE_ZONE3, LOW);
  digitalWrite(HUMID_HIGH_RELAY, LOW);
  digitalWrite(HUMID_LOW_RELAY, LOW);

  // Initialize I2C sensors
  Wire.begin();

  if (!sht31_1.begin(0x44)) {
    Serial.println("SHT31 sensor 1 not found!");
  }
  if (!sht31_2.begin(0x45)) {
    Serial.println("SHT31 sensor 2 not found!");
  }
  if (!scd30.begin()) {
    Serial.println("SCD30 CO2 sensor not found!");
  }

  // Initialize display and touch
  tft.begin();
```

```
  tft.setRotation(1);  // Landscape
  touchDetector.begin();

  Serial.println("System initialized");

  tft.fillScreen(BG_COLOR);
  drawCurrentPage();
}

void drawButton(Button btn, bool pressed = false) {
  uint16_t color = pressed ? BUTTON_PRESS : btn.color;

  tft.fillRoundRect(btn.x, btn.y, btn.w, btn.h, 8, color);
  tft.drawRoundRect(btn.x, btn.y, btn.w, btn.h, 8, TEXT_COLOR);

  tft.setTextColor(TEXT_COLOR);
  tft.setTextSize(4);

  // Center text in button
  int16_t x1, y1;
  uint16_t tw, th;
  tft.getTextBounds(btn.label, 0, 0, &x1, &y1, &tw, &th);
  tft.setCursor(btn.x + (btn.w - tw) / 2, btn.y + (btn.h - th) / 2);
  tft.print(btn.label);
}

bool isTouchInButton(int tx, int ty, Button btn) {
  return (tx >= btn.x && tx <= btn.x + btn.w &&
          ty >= btn.y && ty <= btn.y + btn.h);
}

void drawCurrentPage() {
  tft.fillScreen(BG_COLOR);

  if (currentPage == STATUS_PAGE) {
    drawStatusPageFull();
  } else if (currentPage == SETTINGS_PAGE) {
    drawSettingsPage();
  } else if (currentPage == DEBUG_PAGE) {
    drawDebugPage();
  } else if (currentPage == DIAGRAM_PAGE) {
    drawDiagramPage();
  }

  needsFullRedraw = false;
}

void drawDiagramPage() {
  tft.fillScreen(BG_COLOR);

  // Title
  tft.setTextSize(3);
  tft.setTextColor(TITLE_COLOR);
  tft.setCursor(240, 10);
  tft.println("GREENHOUSE MAP");
```

```
// Page indicator
tft.setTextSize(2);
tft.setTextColor(WARNING_COLOR);
tft.setCursor(10, 10);
tft.print("SW3:Map");

// Draw greenhouse outline
tft.drawRect(50, 60, 700, 390, TEXT_COLOR);
tft.drawRect(52, 62, 696, 386, TEXT_COLOR);

// Draw dividing lines
tft.drawLine(400, 60, 400, 450, TEXT_COLOR);  // Vertical divider
tft.drawLine(50, 255, 750, 255, TEXT_COLOR);   // Horizontal divider

// BOTTOM LEFT ZONE (Zone 0)
int x = 70;
int y = 275;

uint16_t zone0Color = TEXT_COLOR;
if(zones[0].avgMoisture < zones[0].setMoisture - 10) zone0Color = BAD_COLOR;
else if(zones[0].avgMoisture >= zones[0].setMoisture) zone0Color = GOOD_COLOR;
else zone0Color = WARNING_COLOR;

tft.fillRect(x, y, 300, 155, zone0Color);
tft.drawRect(x, y, 300, 155, TEXT_COLOR);

tft.setTextSize(3);
tft.setTextColor(BG_COLOR);
tft.setCursor(x + 50, y + 10);
tft.println("BOTTOM LEFT");

tft.setTextSize(4);
tft.setCursor(x + 70, y + 50);
tft.print((int)zones[0].avgMoisture);
tft.print("%");

tft.setTextSize(2);
tft.setCursor(x + 60, y + 90);
tft.print("Target: ");
tft.print((int)zones[0].setMoisture);
tft.print("%");

// Status
const char* status0 = "";
switch(zones[0].state) {
  case IDLE: status0 = "IDLE"; break;
  case SPRAYING: status0 = "SPRAYING"; break;
  case WAITING: status0 = "WAITING"; break;
}
tft.setCursor(x + 70, y + 120);
tft.print(status0);

// Valve indicator
if(digitalRead(zones[0].valvePin)) {
```

```
  tft.fillCircle(x + 20, y + 20, 10, VALVE_ON_COLOR);
}

// BOTTOM RIGHT ZONE (Zone 1)
x = 420;
y = 275;

uint16_t zone1Color = TEXT_COLOR;
if(zones[1].avgMoisture < zones[1].setMoisture - 10) zone1Color = BAD_COLOR;
else if(zones[1].avgMoisture >= zones[1].setMoisture) zone1Color = GOOD_COLOR;
else zone1Color = WARNING_COLOR;

tft.fillRect(x, y, 300, 155, zone1Color);
tft.drawRect(x, y, 300, 155, TEXT_COLOR);

tft.setTextSize(3);
tft.setTextColor(BG_COLOR);
tft.setCursor(x + 40, y + 10);
tft.println("BOTTOM RIGHT");

tft.setTextSize(4);
tft.setCursor(x + 70, y + 50);
tft.print((int)zones[1].avgMoisture);
tft.print("%");

tft.setTextSize(2);
tft.setCursor(x + 60, y + 90);
tft.print("Target: ");
tft.print((int)zones[1].setMoisture);
tft.print("%");

// Status
const char* status1 = "";
switch(zones[1].state) {
  case IDLE: status1 = "IDLE"; break;
  case SPRAYING: status1 = "SPRAYING"; break;
  case WAITING: status1 = "WAITING"; break;
}
tft.setCursor(x + 70, y + 120);
tft.print(status1);

// Valve indicator
if(digitalRead(zones[1].valvePin)) {
  tft.fillCircle(x + 20, y + 20, 10, VALVE_ON_COLOR);
}

// TOP ZONE (Zone 2)
x = 70;
y = 80;

uint16_t zone2Color = TEXT_COLOR;
if(zones[2].avgMoisture < zones[2].setMoisture - 10) zone2Color = BAD_COLOR;
else if(zones[2].avgMoisture >= zones[2].setMoisture) zone2Color = GOOD_COLOR;
else zone2Color = WARNING_COLOR;
```

```
  tft.fillRect(x, y, 650, 155, zone2Color);
  tft.drawRect(x, y, 650, 155, TEXT_COLOR);

  tft.setTextSize(3);
  tft.setTextColor(BG_COLOR);
  tft.setCursor(x + 270, y + 10);
  tft.println("TOP SHELF");

  tft.setTextSize(4);
  tft.setCursor(x + 280, y + 50);
  tft.print((int)zones[2].avgMoisture);
  tft.print("%");

  tft.setTextSize(2);
  tft.setCursor(x + 250, y + 90);
  tft.print("Target: ");
  tft.print((int)zones[2].setMoisture);
  tft.print("%");

  // Status
  const char* status2 = "";
  switch(zones[2].state) {
    case IDLE: status2 = "IDLE"; break;
    case SPRAYING: status2 = "SPRAYING"; break;
    case WAITING: status2 = "WAITING"; break;
  }
  tft.setCursor(x + 280, y + 120);
  tft.print(status2);

  // Valve indicator
  if(digitalRead(zones[2].valvePin)) {
    tft.fillCircle(x + 20, y + 20, 10, VALVE_ON_COLOR);
  }

  // Environment info at bottom - moved up to prevent cutoff
  tft.setTextSize(2);
  tft.setTextColor(TEXT_COLOR);
  tft.setCursor(60, 458);
  tft.print("Temp: ");
  tft.print((temp1 + temp2) / 2.0, 1);
  tft.print("C  |  Humid: ");
  tft.print((int)((humidity1 + humidity2) / 2.0));
  tft.print("%  |  CO2: ");
  tft.print((int)co2);
  tft.print("ppm");
}

void drawDebugPage() {
  tft.fillScreen(BG_COLOR);

  // Title
  tft.setTextSize(2);
  tft.setTextColor(TITLE_COLOR);
  tft.setCursor(320, 5);
  tft.println("DEBUG MODE");
```

```
// Page indicator
tft.setTextSize(1);
tft.setTextColor(WARNING_COLOR);
tft.setCursor(10, 5);
tft.print("SW0:Debug");

int y = 30;
int col1X = 10;
int col2X = 280;
int col3X = 550;

tft.setTextSize(2);

// COLUMN 1 - Soil Sensors
tft.setTextColor(TITLE_COLOR);
tft.setCursor(col1X, y);
tft.println("SOIL RAW:");
y += 20;

tft.setTextColor(TEXT_COLOR);
for(int i = 0; i < 10; i++) {
  tft.setCursor(col1X, y);
  tft.print("A");
  tft.print(i);
  tft.print(":");
  tft.print(soilRawValues[i]);
  y += 18;
}

y += 10;

// Potentiometers
tft.setTextColor(TITLE_COLOR);
tft.setCursor(col1X, y);
tft.println("POTS:");
y += 20;

tft.setTextColor(TEXT_COLOR);
for(int i = 0; i < 4; i++) {
  tft.setCursor(col1X, y);
  tft.print("P");
  tft.print(i);
  tft.print(": ");
  tft.print((int)potValues[i]);
  tft.print("%");
  y += 18;
}

// COLUMN 2 - Environment & Zones
y = 30;

tft.setTextColor(TITLE_COLOR);
tft.setCursor(col2X, y);
tft.println("ENVIRONMENT:");
```

```
y += 20;

tft.setTextColor(TEXT_COLOR);
tft.setCursor(col2X, y);
tft.print("T1: ");
tft.print(temp1, 1);
tft.print("C");
y += 18;

tft.setCursor(col2X, y);
tft.print("T2: ");
tft.print(temp2, 1);
tft.print("C");
y += 18;

tft.setCursor(col2X, y);
tft.print("Avg: ");
tft.print((temp1 + temp2) / 2.0, 1);
tft.print("C");
y += 22;

tft.setCursor(col2X, y);
tft.print("H1: ");
tft.print(humidity1, 1);
tft.print("%");
y += 18;

tft.setCursor(col2X, y);
tft.print("H2: ");
tft.print(humidity2, 1);
tft.print("%");
y += 18;

tft.setCursor(col2X, y);
tft.print("Avg: ");
tft.print((humidity1 + humidity2) / 2.0, 1);
tft.print("%");
y += 18;

tft.setCursor(col2X, y);
tft.print("Set: ");
tft.print((int)setHumidity);
tft.print("%");
y += 22;

tft.setCursor(col2X, y);
tft.print("CO2: ");
tft.print((int)co2);
y += 25;

// Zone info
tft.setTextColor(TITLE_COLOR);
tft.setCursor(col2X, y);
tft.println("ZONES:");
y += 20;
```

```
tft.setTextColor(TEXT_COLOR);
for(int i = 0; i < 3; i++) {
  tft.setCursor(col2X, y);
  tft.print("Z");
  tft.print(i+1);
  tft.print(": ");
  tft.print((int)zones[i].avgMoisture);
  tft.print("/");
  tft.print((int)zones[i].setMoisture);
  y += 18;
}

// COLUMN 3 - System Status
y = 30;

tft.setTextColor(TITLE_COLOR);
tft.setCursor(col3X, y);
tft.println("SWITCHES:");
y += 20;

for(int i = 0; i < 4; i++) {
  int switchVal = digitalRead(10 + i);
  bool switchOn = (switchVal == LOW);

  tft.setTextColor(switchOn ? GOOD_COLOR : BAD_COLOR);
  tft.setCursor(col3X, y);
  tft.print("SW");
  tft.print(i);
  tft.print(":");
  tft.print(switchOn ? "ON" : "OFF");
  y += 18;
}

y += 10;

// Valves
tft.setTextColor(TITLE_COLOR);
tft.setCursor(col3X, y);
tft.println("VALVES:");
y += 20;

for(int i = 0; i < 3; i++) {
  bool valveOn = digitalRead(zones[i].valvePin);
  tft.setTextColor(valveOn ? VALVE_ON_COLOR : TEXT_COLOR);
  tft.setCursor(col3X, y);
  tft.print("V");
  tft.print(i+1);
  tft.print(":");
  tft.print(valveOn ? "ON" : "OFF");
  y += 18;
}

y += 10;
```

```
// Humidity Control
tft.setTextColor(TITLE_COLOR);
tft.setCursor(col3X, y);
tft.println("HUM CTRL:");
y += 20;

bool dehum = digitalRead(HUMID_HIGH_RELAY);
bool hum = digitalRead(HUMID_LOW_RELAY);

tft.setTextColor(dehum ? VALVE_ON_COLOR : TEXT_COLOR);
tft.setCursor(col3X, y);
tft.print("DeH:");
tft.print(dehum ? "ON" : "OFF");
y += 18;

tft.setTextColor(hum ? VALVE_ON_COLOR : TEXT_COLOR);
tft.setCursor(col3X, y);
tft.print("Hum:");
tft.print(hum ? "ON" : "OFF");
y += 25;

// Irrigation States
tft.setTextColor(TITLE_COLOR);
tft.setCursor(col3X, y);
tft.println("IRRIG:");
y += 20;

for(int i = 0; i < 3; i++) {
  const char* status = "";
  uint16_t color = TEXT_COLOR;

  switch(zones[i].state) {
    case IDLE: status = "IDLE"; color = TEXT_COLOR; break;
    case SPRAYING: status = "SPRAY"; color = VALVE_ON_COLOR; break;
    case WAITING: status = "WAIT"; color = WARNING_COLOR; break;
  }

  tft.setTextColor(color);
  tft.setCursor(col3X, y);
  tft.print("Z");
  tft.print(i+1);
  tft.print(":");
  tft.print(status);

  if (zones[i].state != IDLE) {
    unsigned long elapsed = millis() - zones[i].stateStartTime;
    unsigned long remaining;

    if (zones[i].state == SPRAYING) {
      remaining = (SPRAY_DURATION - elapsed) / 1000;
    } else {
      remaining = (WAIT_DURATION - elapsed) / 1000;
    }

    tft.print(" ");
```

```
      tft.print(remaining);
      tft.print("s");
    }

    y += 18;
  }

  y += 15;

  // System Info
  tft.setTextColor(TITLE_COLOR);
  tft.setCursor(col3X, y);
  tft.println("SYSTEM:");
  y += 20;

  tft.setTextColor(TEXT_COLOR);
  tft.setCursor(col3X, y);
  tft.print("Up:");
  tft.print(millis() / 1000);
  tft.print("s");
}

void drawStatusPageFull() {
  tft.fillScreen(BG_COLOR);

  // Title
  tft.setTextSize(4);
  tft.setTextColor(TITLE_COLOR);
  tft.setCursor(150, 10);
  tft.println("GREENHOUSE STATUS");

  // Page indicator
  tft.setTextSize(2);
  tft.setTextColor(WARNING_COLOR);
  tft.setCursor(10, 10);
  tft.print("SW1:Set");
  tft.setCursor(10, 30);
  tft.print("SW3:Map");

  // Left Column
  int y = 80;

  // Zone moisture
  tft.setTextSize(3);
  tft.setTextColor(WARNING_COLOR);
  tft.setCursor(10, y);
  tft.println("ZONES:");
  y += 30;

  for(int i = 0; i < 3; i++) {
    uint16_t color = TEXT_COLOR;
    if(zones[i].avgMoisture < zones[i].setMoisture - 10) {
      color = BAD_COLOR;
    } else if(zones[i].avgMoisture >= zones[i].setMoisture) {
      color = GOOD_COLOR;
```

```
  } else {
    color = WARNING_COLOR;
  }

  tft.fillRect(10, y, 380, 28, BG_COLOR);
  tft.setTextColor(color);
  tft.setTextSize(3);
  tft.setCursor(10, y);
  tft.print(zones[i].name);
  tft.setCursor(250, y);
  tft.print((int)zones[i].avgMoisture);
  tft.print("/");
  tft.print((int)zones[i].setMoisture);
  y += 35;
}

y += 15;

// Environment
tft.setTextSize(3);
tft.setTextColor(WARNING_COLOR);
tft.setCursor(10, y);
tft.println("ENVIRONMENT:");
y += 30;

tft.fillRect(10, y, 380, 28, BG_COLOR);
tft.setTextColor(TEXT_COLOR);
tft.setTextSize(3);
tft.setCursor(10, y);
tft.print("Temp: ");
tft.print((temp1 + temp2) / 2.0, 1);
tft.print("C");
y += 35;

float avgHum = (humidity1 + humidity2) / 2.0;
uint16_t humColor = TEXT_COLOR;
if(abs(avgHum - setHumidity) > 10) humColor = BAD_COLOR;
else if(abs(avgHum - setHumidity) > 5) humColor = WARNING_COLOR;
else humColor = GOOD_COLOR;

tft.fillRect(10, y, 380, 28, BG_COLOR);
tft.setTextColor(humColor);
tft.setTextSize(3);
tft.setCursor(10, y);
tft.print("Humid: ");
tft.print((int)avgHum);
tft.print("/");
tft.print((int)setHumidity);
y += 35;

tft.fillRect(10, y, 380, 28, BG_COLOR);
tft.setTextColor(TEXT_COLOR);
tft.setTextSize(3);
tft.setCursor(10, y);
tft.print("CO2: ");
```

```
  tft.print((int)co2);

  // Right Column
  y = 80;
  int xRight = 420;

  // Valve Status
  tft.setTextSize(3);
  tft.setTextColor(WARNING_COLOR);
  tft.setCursor(xRight, y);
  tft.println("IRRIGATION:");
  y += 30;

  for(int i = 0; i < 3; i++) {
    const char* status = "";
    uint16_t color = TEXT_COLOR;

    switch(zones[i].state) {
      case IDLE: status = "IDLE"; color = TEXT_COLOR; break;
      case SPRAYING: status = "SPRAY"; color = VALVE_ON_COLOR; break;
      case WAITING: status = "WAIT"; color = WARNING_COLOR; break;
    }

    tft.fillRect(xRight, y, 360, 28, BG_COLOR);
    tft.setTextColor(color);
    tft.setTextSize(3);
    tft.setCursor(xRight, y);
    tft.print(zones[i].name);
    tft.setCursor(xRight + 220, y);
    tft.print(status);
    y += 35;
  }

  y += 25;

  // Humidity control
  tft.fillRect(xRight, y, 360, 28, BG_COLOR);
  tft.setTextColor(TEXT_COLOR);
  tft.setTextSize(3);
  tft.setCursor(xRight, y);
  tft.print("Humid: ");
  if(avgHum > setHumidity + 10) {
    tft.setTextColor(VALVE_ON_COLOR);
    tft.print("DEHUM");
  } else if(avgHum < setHumidity - 10) {
    tft.setTextColor(VALVE_ON_COLOR);
    tft.print("HUMID");
  } else {
    tft.setTextColor(GOOD_COLOR);
    tft.print("OK");
  }
}

void drawSettingsPage() {
  tft.fillScreen(BG_COLOR);
```

```
// Title
tft.setTextSize(3);
tft.setTextColor(TITLE_COLOR);
tft.setCursor(280, 5);
tft.println("SETPOINTS");

// Page indicator
tft.setTextSize(2);
tft.setTextColor(WARNING_COLOR);
tft.setCursor(10, 35);
tft.print("SW0/SW1:Back");

tft.setTextSize(3);

// Bottom Left Zone
int y = 110;
tft.fillRect(160, y + 20, 480, 30, BG_COLOR);
tft.setTextColor(TEXT_COLOR);
tft.setCursor(200, y + 20);
tft.print("Bottom Left: ");
tft.print((int)zones[0].setMoisture);
tft.print("%");
drawButton(zone1MinusBtn);
drawButton(zone1PlusBtn);

// Bottom Right Zone
y = 210;
tft.fillRect(160, y + 20, 480, 30, BG_COLOR);
tft.setCursor(200, y + 20);
tft.print("Bottom Right: ");
tft.print((int)zones[1].setMoisture);
tft.print("%");
drawButton(zone2MinusBtn);
drawButton(zone2PlusBtn);

// Top Zone
y = 310;
tft.fillRect(160, y + 20, 480, 30, BG_COLOR);
tft.setCursor(200, y + 20);
tft.print("Top: ");
tft.print((int)zones[2].setMoisture);
tft.print("%");
drawButton(zone3MinusBtn);
drawButton(zone3PlusBtn);

// Humidity
y = 410;
tft.fillRect(160, y + 20, 480, 30, BG_COLOR);
tft.setCursor(200, y + 20);
tft.print("Humidity: ");
tft.print((int)setHumidity);
tft.print("%");
drawButton(humidMinusBtn);
drawButton(humidPlusBtn);
```

```
}

void handleTouch(int x, int y) {
  // FIXED: Coordinates are swapped AND Y is inverted
  int temp = x;
  x = y;
  y = temp;
  y = 480 - y;

  if (currentPage == SETTINGS_PAGE) {
    // Zone buttons - increment by 1
    if (isTouchInButton(x, y, zone1MinusBtn)) {
      zones[0].setMoisture = max(0.0f, zones[0].setMoisture - 1);
      drawSettingsPage();
      delay(200);
    }
    else if (isTouchInButton(x, y, zone1PlusBtn)) {
      zones[0].setMoisture = min(100.0f, zones[0].setMoisture + 1);
      drawSettingsPage();
      delay(200);
    }
    else if (isTouchInButton(x, y, zone2MinusBtn)) {
      zones[1].setMoisture = max(0.0f, zones[1].setMoisture - 1);
      drawSettingsPage();
      delay(200);
    }
    else if (isTouchInButton(x, y, zone2PlusBtn)) {
      zones[1].setMoisture = min(100.0f, zones[1].setMoisture + 1);
      drawSettingsPage();
      delay(200);
    }
    else if (isTouchInButton(x, y, zone3MinusBtn)) {
      zones[2].setMoisture = max(0.0f, zones[2].setMoisture - 1);
      drawSettingsPage();
      delay(200);
    }
    else if (isTouchInButton(x, y, zone3PlusBtn)) {
      zones[2].setMoisture = min(100.0f, zones[2].setMoisture + 1);
      drawSettingsPage();
      delay(200);
    }
    else if (isTouchInButton(x, y, humidMinusBtn)) {
      setHumidity = max(0.0f, setHumidity - 1);
      drawSettingsPage();
      delay(200);
    }
    else if (isTouchInButton(x, y, humidPlusBtn)) {
      setHumidity = min(100.0f, setHumidity + 1);
      drawSettingsPage();
      delay(200);
    }
  }
}

// Calculate zone 1 average (A0-A3)
```

```
float calculateZone1Average() {
  float readings[4];

  analogRead(A0); delayMicroseconds(100);
  int raw = (analogRead(A0) + analogRead(A0) + analogRead(A0)) / 3;
  soilRawValues[0] = raw;
  readings[0] = map(raw, 0, 4095, 0, 100);

  analogRead(A1); delayMicroseconds(100);
  raw = (analogRead(A1) + analogRead(A1) + analogRead(A1)) / 3;
  soilRawValues[1] = raw;
  readings[1] = map(raw, 0, 4095, 0, 100);

  analogRead(A2); delayMicroseconds(100);
  raw = (analogRead(A2) + analogRead(A2) + analogRead(A2)) / 3;
  soilRawValues[2] = raw;
  readings[2] = map(raw, 0, 4095, 0, 100);

  analogRead(A3); delayMicroseconds(100);
  raw = (analogRead(A3) + analogRead(A3) + analogRead(A3)) / 3;
  soilRawValues[3] = raw;
  readings[3] = map(raw, 0, 4095, 0, 100);

  // Sort to find median
  for(int i = 0; i < 3; i++) {
    for(int j = 0; j < 3-i; j++) {
      if(readings[j] > readings[j+1]) {
        float temp = readings[j];
        readings[j] = readings[j+1];
        readings[j+1] = temp;
      }
    }
  }

  float median = (readings[1] + readings[2]) / 2.0;

  float sum = 0;
  int validCount = 0;
  for(int i = 0; i < 4; i++) {
    if(abs(readings[i] - median) < 30) {
      sum += readings[i];
      validCount++;
    }
  }

  return validCount > 0 ? sum / validCount : median;
}

// Calculate zone 2 average (A4-A7)
float calculateZone2Average() {
  float readings[4];

  analogRead(A4); delayMicroseconds(100);
  int raw = (analogRead(A4) + analogRead(A4) + analogRead(A4)) / 3;
  soilRawValues[4] = raw;
```

```
  readings[0] = map(raw, 0, 4095, 0, 100);

  analogRead(A5); delayMicroseconds(100);
  raw = (analogRead(A5) + analogRead(A5) + analogRead(A5)) / 3;
  soilRawValues[5] = raw;
  readings[1] = map(raw, 0, 4095, 0, 100);

  analogRead(A6); delayMicroseconds(100);
  raw = (analogRead(A6) + analogRead(A6) + analogRead(A6)) / 3;
  soilRawValues[6] = raw;
  readings[2] = map(raw, 0, 4095, 0, 100);

  analogRead(A7); delayMicroseconds(100);
  raw = (analogRead(A7) + analogRead(A7) + analogRead(A7)) / 3;
  soilRawValues[7] = raw;
  readings[3] = map(raw, 0, 4095, 0, 100);

  // Sort to find median
  for(int i = 0; i < 3; i++) {
    for(int j = 0; j < 3-i; j++) {
      if(readings[j] > readings[j+1]) {
        float temp = readings[j];
        readings[j] = readings[j+1];
        readings[j+1] = temp;
      }
    }
  }

  float median = (readings[1] + readings[2]) / 2.0;

  float sum = 0;
  int validCount = 0;
  for(int i = 0; i < 4; i++) {
    if(abs(readings[i] - median) < 30) {
      sum += readings[i];
      validCount++;
    }
  }

  return validCount > 0 ? sum / validCount : median;
}

// Calculate zone 3 average (A8-A9)
float calculateZone3Average() {
  float readings[2];

  analogRead(A8); delayMicroseconds(100);
  int raw = (analogRead(A8) + analogRead(A8) + analogRead(A8)) / 3;
  soilRawValues[8] = raw;
  readings[0] = map(raw, 0, 4095, 0, 100);

  analogRead(A9); delayMicroseconds(100);
  raw = (analogRead(A9) + analogRead(A9) + analogRead(A9)) / 3;
  soilRawValues[9] = raw;
  readings[1] = map(raw, 0, 4095, 0, 100);
```

```
  if(abs(readings[0] - readings[1]) > 30) {
    return min(readings[0], readings[1]);
  }

  return (readings[0] + readings[1]) / 2.0;
}

void readPotValues() {
  for(int i=0; i<5; i++) analogRead(A10);
  delay(10);
  potValues[0] = map(analogRead(A10), 0, 4095, 0, 100);

  for(int i=0; i<5; i++) analogRead(A11);
  delay(10);
  potValues[1] = map(analogRead(A11), 0, 4095, 0, 100);

  for(int i=0; i<5; i++) analogRead(A12);
  delay(10);
  potValues[2] = map(analogRead(A12), 0, 4095, 0, 100);

  for(int i=0; i<5; i++) analogRead(A13);
  delay(10);
  potValues[3] = map(analogRead(A13), 0, 4095, 0, 100);
}

void readEnvironmentSensors() {
  temp1 = sht31_1.readTemperature();
  humidity1 = sht31_1.readHumidity();

  temp2 = sht31_2.readTemperature();
  humidity2 = sht31_2.readHumidity();

  if (scd30.dataReady()) {
    scd30.read();
    co2 = scd30.CO2;
  }
}

void updateIrrigationZone(int zoneIdx) {
  Zone* zone = &zones[zoneIdx];
  unsigned long currentTime = millis();
  unsigned long elapsed = currentTime - zone->stateStartTime;

  switch(zone->state) {
    case IDLE:
      if(zone->avgMoisture < zone->setMoisture) {
        zone->state = SPRAYING;
        zone->stateStartTime = currentTime;
        digitalWrite(zone->valvePin, HIGH);
        Serial.print("Zone "); Serial.print(zoneIdx); Serial.println(" spraying");
      }
      break;

    case SPRAYING:
```

```
      if(elapsed >= SPRAY_DURATION) {
        zone->state = WAITING;
        zone->stateStartTime = currentTime;
        digitalWrite(zone->valvePin, LOW);
        Serial.print("Zone "); Serial.print(zoneIdx); Serial.println(" waiting");
      }
      break;

    case WAITING:
      if(elapsed >= WAIT_DURATION) {
        zone->state = IDLE;
        Serial.print("Zone "); Serial.print(zoneIdx); Serial.println(" checking");
      }
      break;
  }
}

void controlHumidity() {
  float avgHumidity = (humidity1 + humidity2) / 2.0;

  if(avgHumidity > setHumidity + 10) {
    digitalWrite(HUMID_HIGH_RELAY, HIGH);
    digitalWrite(HUMID_LOW_RELAY, LOW);
  }
  else if(avgHumidity < setHumidity - 10) {
    digitalWrite(HUMID_HIGH_RELAY, LOW);
    digitalWrite(HUMID_LOW_RELAY, HIGH);
  }
  else {
    digitalWrite(HUMID_HIGH_RELAY, LOW);
    digitalWrite(HUMID_LOW_RELAY, LOW);
  }
}

void loop() {
  // Check switches for page selection
  int sw0 = digitalRead(10);
  int sw1 = digitalRead(11);
  int sw3 = digitalRead(13);

  Page desiredPage;
  if (sw0 == LOW) {
    desiredPage = DEBUG_PAGE;
  } else if (sw1 == LOW) {
    desiredPage = SETTINGS_PAGE;
  } else if (sw3 == LOW) {
    desiredPage = DIAGRAM_PAGE;
  } else {
    desiredPage = STATUS_PAGE;
  }

  if (desiredPage != currentPage) {
    currentPage = desiredPage;
    needsFullRedraw = true;
    drawCurrentPage();
```

```
  }

  // Check for touch
  GDTpoint_t points[5];
  uint8_t contacts = touchDetector.getTouchPoints(points);

  if (contacts > 0) {
    int x = points[0].x;
    int y = points[0].y;
    handleTouch(x, y);
    delay(100);
  }

  // Read all sensors
  static unsigned long lastSensorRead = 0;
  if (millis() - lastSensorRead >= 2000) {
    zones[0].avgMoisture = calculateZone1Average();
    zones[1].avgMoisture = calculateZone2Average();
    zones[2].avgMoisture = calculateZone3Average();

    readPotValues();
    readEnvironmentSensors();

    // Update irrigation
    for(int i = 0; i < 3; i++) {
      updateIrrigationZone(i);
    }

    // Control humidity
    controlHumidity();

    // Redraw debug or diagram pages
    if (currentPage == DEBUG_PAGE || currentPage == DIAGRAM_PAGE || needsFullRedraw) {
      drawCurrentPage();
    }

    lastSensorRead = millis();
  }
}
```